OpsPilot — Complete Solution

> The whole project, every file — nothing to copy from anywhere else. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck. All of this code runs against the example services with no external setup.

The final layout

```
<repo folder>/
├── servers/
│   ├── runbook_server.py
│   └── status_server.py
├── src/
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── agent.py
│   ├── config.py
│   ├── exceptions.py
│   ├── server_pool.py
│   ├── servers_config.py
│   └── tool_bridge.py
├── tests/
│   ├── integration/
│   │   └── test_agent.py
│   └── unit/
│       ├── test_config.py
│       └── test_tool_bridge.py
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
└── requirements.txt
```

The project files you make once

`requirements.txt`

```
mcp
openai
python-dotenv
pydantic
```

`.env.example` (copy to `.env`, put your real key in `.env`)

```
OPENAI_API_KEY=sk-your-key-here
LOG_LEVEL=INFO
MCP_CONNECT_TIMEOUT=10
MCP_MAX_ITERATIONS=6
```

`.gitignore`

```
.env
.venv/
__pycache__/
*.pyc
```

Every folder under `src/` gets an **empty** `__init__.py`: `src/__init__.py`, `src/models/__init__.py`, `src/utils/__init__.py`. They hold nothing — the file just needs to exist so Python treats the folder as a package.

`LICENSE` — MIT (fill in the year and your name):

```
MIT License

Copyright (c) 2026 <your name>

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
```

```
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

The two example services

```
# servers/status_server.py
# A minimal MCP service for service health. The agent launches this as a
# subprocess and discovers its tools — it never imports this file.
import sys
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("status-server")

STATUS = {
    "checkout": "DOWN since 14:02 UTC - 503 rate 88%.",
    "search": "healthy - p95 210ms.",
}
DEPLOYS = {
    "checkout": "v812 deployed 13:58 UTC by sara (14 commits).",
    "search": "v77 deployed yesterday.",
}


@mcp.tool()
def get_service_status(service: str) -> str:
    """Get the current health status of one service by name."""
    if service not in STATUS:
        return f"Error: no service named {service}"
    return STATUS[service]


@mcp.tool()
def recent_deploys(service: str) -> str:
    """Get the most recent deploy for one service by name."""
    if service not in DEPLOYS:
        return f"Error: no service named {service}"
    return DEPLOYS[service]


if __name__ == "__main__":
```

```
    print("status server starting", file=sys.stderr)
    mcp.run(transport="stdio")
```

```
# servers/runbook_server.py
# A minimal MCP service for runbooks. Independent of the status service —
# its own domain, its own tools.
import sys
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("runbook-server")

RUNBOOKS = {
    "rb-checkout": "Checkout 503s: roll back the deploy, then page payments.",
    "rb-search": "Search latency: clear the query cache and add replicas.",
}


@mcp.tool()
def search_runbooks(query: str) -> str:
    """Search runbooks by keyword; return matching runbook ids."""
    hits = []
    for runbook_id, text in RUNBOOKS.items():
        if query.lower() in text.lower() or query.lower() in runbook_id:
            hits.append(f"{runbook_id}: {text[:30]}")
    if not hits:
        return "No matching runbooks."
    return "\n".join(hits)


@mcp.tool()
def read_runbook(runbook_id: str) -> str:
    """Read the full text of one runbook by its id."""
    if runbook_id not in RUNBOOKS:
        return f"Error: no runbook with id {runbook_id}"
    return RUNBOOKS[runbook_id]


if __name__ == "__main__":
    print("runbook server starting", file=sys.stderr)
    mcp.run(transport="stdio")
```

Settings, errors, logging

```
# src/exceptions.py
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass


class ServerUnreachableError(Exception):
    """Raised when an MCP service cannot be connected to in time."""
    pass


class MaxIterationsExceeded(Exception):
    """Raised when the ReAct loop runs more steps than its hard limit."""
    pass
```

```
# src/config.py
# Loads settings from .env into a checked Config object (the Doc01 pattern).
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError


@dataclass
class Config:
    openai_api_key: str
    log_level: str
    connect_timeout: float
    max_iterations: int


def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value


def load_config() -> Config:
    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")
    log_level = os.getenv("LOG_LEVEL", "INFO")
    connect_timeout = float(os.getenv("MCP_CONNECT_TIMEOUT", "10"))
    max_iterations = int(os.getenv("MCP_MAX_ITERATIONS", "6"))
```

```
    return Config(
        openai_api_key=api_key,
        log_level=log_level,
        connect_timeout=connect_timeout,
        max_iterations=max_iterations,
    )
```

```
# src/utils/logger.py
# One logging setup for every file. Logs to STDERR on purpose: an MCP
# client can carry protocol traffic on stdout.
import logging
import os
import sys


def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler(sys.stderr)
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

The schema bridge

```
# src/tool_bridge.py
# Adapters between MCP's shapes and OpenAI's, so the loop never has to know
# either library's exact types.
from src.utils.logger import get_logger


def one_openai_tool(namespaced_name, description, input_schema):
    return {
        "type": "function",
        "function": {
            "name": namespaced_name,
            "description": description,
            "parameters": input_schema,
        },
```

```
    }


def mcp_tools_to_openai_schema(named_tools):
    logger = get_logger(__name__)
    schemas = []
    for namespaced_name, tool in named_tools:
        description = tool.description
        if description is None:
            description = ""
        schemas.append(
            one_openai_tool(namespaced_name, description, tool.input_schema)
        )
    logger.info(f"converted {len(schemas)} MCP tools to OpenAI schema")
    return schemas


def extract_text(call_result):
    parts = []
    for block in call_result.content:
        text = getattr(block, "text", None)
        if text is not None:
            parts.append(text)
    return "\n".join(parts)
```

The server pool (connect, discover, route, survive)

```
# src/server_pool.py
# MCPServerPool: connect to every configured service, discover and namespace
# its tools, and route calls back to the right service. Resilient: a service
# that can't connect is skipped; a tool call that fails mid-task returns a
# clear error string instead of crashing the loop.
import asyncio
from contextlib import AsyncExitStack

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

from src.tool_bridge import extract_text, mcp_tools_to_openai_schema
from src.utils.logger import get_logger

NAME_SEPARATOR = "__"


class MCPServerPool:
```

```
    def __init__(self, servers, connect_timeout=10.0):
        self.servers = servers
        self.connect_timeout = connect_timeout
        self.stack = AsyncExitStack()
        self.sessions = {}
        self.routing = {}
        self.named_tools = []
        self.logger = get_logger(__name__)

    async def _connect_one(self, server):
        # The context managers are entered in THIS (the main) task, never
        # inside asyncio.wait_for: wait_for runs its coroutine in a new
        # task, and a context entered in one task cannot be closed in
        # another. Only initialize() -- which can hang -- is timed out.
        params = StdioServerParameters(
            command=server["command"], args=server["args"]
        )
        read, write = await self.stack.enter_async_context(
            stdio_client(params)
        )
        session = await self.stack.enter_async_context(
            ClientSession(read, write)
        )
        await asyncio.wait_for(
            session.initialize(), timeout=self.connect_timeout
        )
        return session

    async def connect_all(self):
        for server in self.servers:
            name = server["name"]
            try:
                session = await self._connect_one(server)
            except Exception as e:
                self.logger.warning(
                    f"[{name}] unreachable: {e}, continuing without it"
                )
                continue
            self.sessions[name] = session
            listed = await session.list_tools()
            for tool in listed.tools:
                namespaced = name + NAME_SEPARATOR + tool.name
                self.routing[namespaced] = (name, tool.name)
                self.named_tools.append((namespaced, tool))
            self.logger.info(f"[{name}] connected, {len(listed.tools)} tools")

    def openai_tools(self):
```

```
        return mcp_tools_to_openai_schema(self.named_tools)

    async def call_tool(self, namespaced_name, arguments):
        if namespaced_name not in self.routing:
            return f"Error: no tool named {namespaced_name} is available"
        server_name, tool_name = self.routing[namespaced_name]
        session = self.sessions[server_name]
        try:
            result = await session.call_tool(tool_name, arguments)
        except Exception as e:
            self.logger.warning(f"[{server_name}] call failed: {e}")
            return f"Error: tool {namespaced_name} failed: {e}"
        return extract_text(result)

    async def aclose(self):
        await self.stack.aclose()
```

> **The one real async trap:** do not write `await asyncio.wait_for(self._connect_one(server), ...)`. `wait_for` runs its coroutine in a **new task**, so the `stdio_client` / `ClientSession` contexts get entered there; closing them later in the main task raises `RuntimeError: Attempted to exit cancel scope in a different task`. Enter the contexts in the main task, and time out only `initialize()`.

The OpenAI call and the ReAct loop

```
# src/models/llm.py
# The only file that talks to the OpenAI SDK.
import openai
from openai import OpenAI

from src.config import Config

MODEL = "gpt-4o-mini"


def create_client(config: Config) -> OpenAI:
    return OpenAI(api_key=config.openai_api_key)


def check_api_key_at_startup(client: OpenAI) -> bool:
    try:
        client.chat.completions.create(
            model=MODEL,
```

```
            messages=[{"role": "user", "content": "ping"}],
            max_tokens=1,
        )
    except openai.AuthenticationError:
        return False
    return True


def ask_model(client, messages, tools):
    response = client.chat.completions.create(
        model=MODEL, messages=messages, tools=tools
    )
    return response.choices[0].message
```

```
# src/agent.py
# The ReAct loop: think (ask the model) -> act (call a discovered tool via
# the pool) -> observe (feed the result back). Bounded by max_iterations.
import json
from dataclasses import dataclass

from src.exceptions import MaxIterationsExceeded
from src.models.llm import ask_model
from src.utils.logger import get_logger


@dataclass
class AgentResult:
    answer: str
    steps: list


def build_messages(task):
    system = (
        "You are an on-call assistant. Use the available tools to answer "
        "the question. When you have enough information, answer plainly."
    )
    return [
        {"role": "system", "content": system},
        {"role": "user", "content": task},
    ]


async def run_agent(task, pool, client, max_iterations=6):
    logger = get_logger(__name__)
    messages = build_messages(task)
    tools = pool.openai_tools()
```

```
    steps = []

    for step_number in range(max_iterations):
        reply = ask_model(client, messages, tools)
        if not reply.tool_calls:
            logger.info("agent finished with a final answer")
            return AgentResult(answer=reply.content, steps=steps)

        messages.append({
            "role": "assistant",
            "content": reply.content,
            "tool_calls": _tool_calls_as_dicts(reply.tool_calls),
        })
        for call in reply.tool_calls:
            name = call.function.name
            arguments = json.loads(call.function.arguments)
            logger.info(f"step {step_number}: calling {name} {arguments}")
            observation = await pool.call_tool(name, arguments)
            steps.append({"tool": name, "args": arguments,
                          "observation": observation})
            messages.append({
                "role": "tool",
                "tool_call_id": call.id,
                "content": observation,
            })

    raise MaxIterationsExceeded(
        f"agent did not finish within {max_iterations} steps"
    )


def _tool_calls_as_dicts(tool_calls):
    out = []
    for call in tool_calls:
        out.append({
            "id": call.id,
            "type": "function",
            "function": {
                "name": call.function.name,
                "arguments": call.function.arguments,
            },
        })
    return out
```

The config and the entry point

```
# src/servers_config.py
# The services to connect to, as plain data. OpsPilot never hardcodes these
# services' TOOLS — only how to launch them.
SERVERS = [
    {"name": "status", "command": "python",
     "args": ["servers/status_server.py"]},
    {"name": "runbooks", "command": "python",
     "args": ["servers/runbook_server.py"]},
]
```

```
# main.py
# The demo you run: connect to the configured services, discover their
# tools, answer one on-call question, and print each step.
import asyncio
import sys

from src.agent import run_agent
from src.config import load_config
from src.exceptions import MissingConfigError
from src.models.llm import create_client
from src.server_pool import MCPServerPool
from src.servers_config import SERVERS

TASK = ("Is checkout down right now, and what does the runbook say to do "
        "about it?")


async def run_demo():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    client = create_client(config)
    pool = MCPServerPool(SERVERS, connect_timeout=config.connect_timeout)
    await pool.connect_all()
    print(f"Connected servers: {list(pool.sessions.keys())}")
    print(f"Tools discovered: {len(pool.named_tools)}")
    try:
        print(f"\nTask: {TASK}")
        result = await run_agent(
            TASK, pool, client, max_iterations=config.max_iterations
        )
        for step in result.steps:
            print(f"  -> {step['tool']} {step['args']}")
```

```
        print(f"Answer: {result.answer}")
    finally:
        await pool.aclose()


def main():
    asyncio.run(run_demo())


if __name__ == "__main__":
    main()
```

Expected output (`python main.py` , with a real key) — the incident question routes to both services:

```
Connected servers: ['status', 'runbooks']
Tools discovered: 4

Task: Is checkout down right now, and what does the runbook say to do about it?
  -> status__get_service_status {'service': 'checkout'}
  -> runbooks__search_runbooks {'query': 'checkout'}
  -> runbooks__read_runbook {'runbook_id': 'rb-checkout'}
Answer: Checkout has been down since 14:02 (503 rate 88%). The runbook says
to roll back the deploy, then page payments.
```

The tests

```
# tests/unit/test_config.py
# Checks the settings code with no network. Each case prints OK or WRONG.
# It changes only a made-up setting, so your real .env is never touched.
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "OPSPILOT_TEST_SETTING"


def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
```

```
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")


def test_empty_setting() -> None:
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")


def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    if config.max_iterations <= 0:
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")


if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

```
# tests/unit/test_tool_bridge.py
# Checks the schema bridge with tiny fakes — no service, no network.
# Run from the repo root: python -m tests.unit.test_tool_bridge
from src.tool_bridge import extract_text, mcp_tools_to_openai_schema


class FakeTool:
    def __init__(self, name, description, input_schema):
        self.name = name
        self.description = description
        self.input_schema = input_schema


class FakeBlock:
    def __init__(self, text):
```

```
        self.text = text


class FakeResult:
    def __init__(self, texts):
        self.content = []
        for text in texts:
            self.content.append(FakeBlock(text))


def test_schema_shape() -> None:
    tool = FakeTool("get_service_status", "Get status.",
                    {"type": "object", "properties": {}})
    schemas = mcp_tools_to_openai_schema([("status__get_service_status", tool)])
    one = schemas[0]
    result = "OK"
    if one["type"] != "function":
        result = "WRONG"
    if one["function"]["name"] != "status__get_service_status":
        result = "WRONG"
    if "properties" not in one["function"]["parameters"]:
        result = "WRONG"
    print(f"case 1 (openai schema shape)   -> {result}")


def test_missing_description() -> None:
    tool = FakeTool("t", None, {"type": "object"})
    schemas = mcp_tools_to_openai_schema([("s__t", tool)])
    result = "OK"
    if schemas[0]["function"]["description"] != "":
        result = "WRONG"
    print(f"case 2 (None description -> '') -> {result}")


def test_extract_text_joins() -> None:
    result_obj = FakeResult(["line one", "line two"])
    text = extract_text(result_obj)
    result = "OK"
    if text != "line one\nline two":
        result = "WRONG"
    print(f"case 3 (extract_text joins)    -> {result}")


if __name__ == "__main__":
    test_schema_shape()
    test_missing_description()
    test_extract_text_joins()
```

```
# tests/integration/test_agent.py
# Launches the real example services over MCP. Each case prints OK or WRONG.
# Run from the repo root: python -m tests.integration.test_agent
import asyncio

from src.agent import run_agent
from src.config import load_config
from src.models.llm import create_client
from src.server_pool import MCPServerPool

STATUS = {"name": "status", "command": "python",
          "args": ["servers/status_server.py"]}
RUNBOOKS = {"name": "runbooks", "command": "python",
            "args": ["servers/runbook_server.py"]}
BROKEN = {"name": "status", "command": "python",
          "args": ["servers/does_not_exist.py"]}


async def connect(servers):
    config = load_config()
    client = create_client(config)
    pool = MCPServerPool(servers, connect_timeout=config.connect_timeout)
    await pool.connect_all()
    return config, client, pool


async def test_discovery_namespaced() -> None:
    config, client, pool = await connect([STATUS, RUNBOOKS])
    names = []
    for namespaced, tool in pool.named_tools:
        names.append(namespaced)
    result = "OK"
    if "status__get_service_status" not in names:
        result = "WRONG"
    if "runbooks__search_runbooks" not in names:
        result = "WRONG"
    await pool.aclose()
    print(f"case 1 (tools discovered + namespaced) -> {result}")


async def test_routing_across_servers() -> None:
    config, client, pool = await connect([STATUS, RUNBOOKS])
    task = ("Is checkout down right now, and what does the runbook say to "
            "do about it?")
    out = await run_agent(task, pool, client, config.max_iterations)
    tools_used = []
    for step in out.steps:
```

```
        tools_used.append(step["tool"])
    result = "OK"
    if "status__get_service_status" not in tools_used:
        result = "WRONG"
    if "runbooks__read_runbook" not in tools_used:
        result = "WRONG"
    await pool.aclose()
    print(f"case 2 (routes to both services)       -> {result}")


async def test_fallback_when_status_down() -> None:
    config, client, pool = await connect([BROKEN, RUNBOOKS])
    result = "OK"
    if "status" in pool.sessions:
        result = "WRONG"
    if "runbooks" not in pool.sessions:
        result = "WRONG"
    out = await run_agent("Is checkout down, and what does the runbook say?",
                          pool, client, config.max_iterations)
    if not out.answer:
        result = "WRONG"
    await pool.aclose()
    print(f"case 3 (status down, runbooks only)    -> {result}")


async def main() -> None:
    await test_discovery_namespaced()
    await test_routing_across_servers()
    await test_fallback_when_status_down()


if __name__ == "__main__":
    asyncio.run(main())
```

Expected unit output (all `OK`):

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

```
case 1 (openai schema shape)   -> OK
case 2 (None description -> '') -> OK
case 3 (extract_text joins)    -> OK
```

Expected integration output (launches the real services over MCP):

```
case 1 (tools discovered + namespaced) -> OK
case 2 (routes to both services)       -> OK
case 3 (status down, runbooks only)    -> OK
```

That's the whole project. If your version discovers every tool from the services (none hardcoded), routes the incident question to both services, and still answers when the status service is down — you've built it.